

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Coding Challenge)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT1-ANALYTICAL PROBLEM SOLVING (ABQ)
Question	Q1 — Writing a Program: Student Grade Book Application (10 Marks)

Q1. (Writing a Program) [10 Marks]

You are building a Student Grade Book application. Write a Python program that:

- (a) Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
- (b) Computes total, average, and grade (O: ≥ 90 , A+: ≥ 80 , A: ≥ 70 , B+: ≥ 60 , B: ≥ 50 , F: < 50) for each student.
- (c) Finds the class topper (highest average) and the subject with the highest overall mean.
- (d) Displays a formatted result table sorted by average in descending order.

Skills Assessed

- Dictionaries, lists, and nested data structures
- Loops, conditionals, and function decomposition
- Aggregate calculations (sum, average, max)
- Sorting with sorted() and formatted string output

Sample Input / Output

Sample Input: gradebook = {"Arun": [92,88,95,90,85], "Divya": [78,82,75,80,88], "Karthik": [55,60,45,50,65], "Meena": [70,68,72,75,74], "Ravi": [40,55,48,52,45]}

Expected Output (see Sample Output section below).

Approach / Design

- **Storing data** (gradebook dictionary): each student's name is a key mapped to a list of five subject marks, matching the required key = name, value = list of 5 subject marks structure.
- **Grading** (compute_grade): applies the cutoff scale (O: ≥ 90 , A+: ≥ 80 , A: ≥ 70 , B+: ≥ 60 , B: ≥ 50 , F: < 50) to a student's average using a simple if/elif chain.
- **Per-student results** (compute_student_results): iterates over the gradebook, computing each student's total with sum(), average by dividing by the number of subjects, and grade via compute_grade, storing everything in a results dictionary.
- **Topper and best subject** (find_topper_and_best_subject): uses max() with a key function to find the student with the highest average, and accumulates per-subject totals across all students to compute each subject's mean, then picks the subject with the highest mean.
- **Display** (display_result_table): sorts the results with sorted(..., reverse=True) on average marks and prints a formatted table with aligned columns for marks, total, average, and grade.

- **Main driver** (main): defines the subject list and gradebook, then chains the computation, topper/subject lookup, and table-display functions together.

Python Program

```
"""
C03-AT1 - Analytical Problem Solving (ABQ)
Q1: Writing a Program - Student Grade Book [10 Marks]
-----
Builds a Student Grade Book application that:
(a) Stores each student's name and marks in a dictionary
    (key = name, value = list of 5 subject marks).
(b) Computes total, average, and grade for each student.
(c) Finds the class topper (highest average) and the subject
    with the highest overall mean.
(d) Displays a formatted result table sorted by average
    in descending order.
"""

# -----
# 1. Grade computation function
# -----
def compute_grade(average):
    """
    Returns the letter grade for a given average based on the
    scale: 0: >=90, A+: >=80, A: >=70, B+: >=60, B: >=50, F: <50
    """
    if average >= 90:
        return "O"
    elif average >= 80:
        return "A+"
    elif average >= 70:
        return "A"
    elif average >= 60:
        return "B+"
    elif average >= 50:
        return "B"
    else:
        return "F"

# -----
# 2. Compute total, average, and grade for every student
# -----
def compute_student_results(gradebook):
    """
    Takes the gradebook dictionary (name -> list of 5 marks) and
    returns a dictionary of results:
    name -> {"marks", "total", "average", "grade"}
    """
    results = {}
    for name, marks in gradebook.items():
        total = sum(marks)
        average = total / len(marks)
        grade = compute_grade(average)
        results[name] = {
            "marks": marks,
            "total": total,
            "average": average,
            "grade": grade,
        }
    return results

# -----
# 3. Find the class topper and the subject with the highest mean
# -----
def find_topper_and_best_subject(results, subjects):
    """
```

```

Finds the student with the highest average (class topper) and
the subject with the highest overall mean across all students.
"""
topper = max(results.items(), key=lambda item: item[1]["average"])
topper_name = topper[0]

subject_totals = [0] * len(subjects)
student_count = len(results)
for data in results.values():
    for i, mark in enumerate(data["marks"]):
        subject_totals[i] += mark

subject_means = [total / student_count for total in subject_totals]
best_subject_index = subject_means.index(max(subject_means))
best_subject_name = subjects[best_subject_index]

return topper_name, best_subject_name, subject_means

# -----
# 4. Display a formatted result table sorted by average (desc)
# -----
def display_result_table(results, subjects):
    """
    Prints a formatted table of all students sorted by average
    marks in descending order.
    """
    header = f"{'Name':<10}" + "".join(f"{'s':<8}" for s in subjects) + \
        f"{'Total':<8}{'Average':<10}{'Grade':<6}"
    print(header)
    print("-" * len(header))

    sorted_students = sorted(
        results.items(), key=lambda item: item[1]["average"], reverse=True
    )
    for name, data in sorted_students:
        row = f"{name:<10}" + "".join(f"{m:<8}" for m in data["marks"]) + \
            f"{data['total']:<8}{data['average']:<10.2f}{data['grade']:<6}"
        print(row)

# -----
# 5. Main driver
# -----
def main():
    subjects = ["Tamil", "English", "Maths", "Science", "Social"]

    # (a) Dictionary: name -> list of 5 subject marks
    gradebook = {
        "Arun": [92, 88, 95, 90, 85],
        "Divya": [78, 82, 75, 80, 88],
        "Karthik": [55, 60, 45, 50, 65],
        "Meena": [70, 68, 72, 75, 74],
        "Ravi": [40, 55, 48, 52, 45],
    }

    # (b) Compute total, average, and grade for each student
    results = compute_student_results(gradebook)

    # (c) Find the class topper and subject with highest overall mean
    topper_name, best_subject, subject_means = find_topper_and_best_subject(
        results, subjects
    )

    # (d) Display formatted result table sorted by average (descending)
    print("Student Grade Book")
    print("=" * 60)
    display_result_table(results, subjects)

    print("\nClass Topper:", topper_name,
        f"(Average: {results[topper_name]['average']:.2f})")

```

```

        print("Subject with Highest Overall Mean:", best_subject,
              f"(Mean: {max(subject_means):.2f})")

if __name__ == "__main__":
    main()

```

Sample Output

```

Student Grade Book
=====
Name      Tamil   English Maths   Science Social   Total   Average   Grade
-----
Arun      92      88      95      90      85      450      90.00      O
Divya     78      82      75      80      88      403      80.60      A+
Meena     70      68      72      75      74      359      71.80      A
Karthik   55      60      45      50      65      275      55.00      B
Ravi      40      55      48      52      45      240      48.00      F

Class Topper: Arun (Average: 90.00)
Subject with Highest Overall Mean: Social (Mean: 71.40)

```

Conclusion

The program models each student's marks as a dictionary entry, then derives total, average, and grade in a single helper function so the logic stays reusable and readable. The class topper is found with `max()` on average marks, while per-subject totals are accumulated across all students to identify the subject with the highest overall mean. Finally, `sorted()` with `reverse=True` produces a results table ranked from the highest to the lowest average, formatted with aligned columns for easy reading. Running the program on the sample gradebook correctly identifies Arun as the class topper with a 90.00 average and Social as the subject with the highest overall mean.